Experiment No 02

AIM: Create a Blockchain using Python

Theory:-

SHA-256 Hash Generation:

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hashing algorithm that converts input data of any size into a fixed 256-bit (64 hexadecimal characters) hash. Even a small change in the input produces a completely different hash, making it useful for data integrity and blockchain security.

Target Hash Generation with Nonce:

In blockchain mining, a nonce is a number that is repeatedly changed and combined with the input data before generating the hash. The purpose is to find a hash that satisfies a particular condition, such as starting with a specified number of zeros.

Proof-of-Work Puzzle Solving:

Proof-of-Work (PoW) is a consensus mechanism used by blockchains such as Bitcoin. Miners repeatedly try different nonce values until they find one that produces a hash meeting the required difficulty target, usually represented by a specific number of leading zeros. This process requires computational effort and helps secure the blockchain.

Merkle Tree Construction:

A Merkle Tree is a tree structure used to efficiently verify the integrity of transactions in a block. Individual transactions are first hashed, then pairs of hashes are combined and hashed again until a single hash, called the Merkle Root, is obtained. If there is an odd number of hashes at a level, the last hash is duplicated. The Merkle Root provides a compact representation of all transactions and helps detect any modification to transaction data.

Tasks Performed:

1. **Hash Generation using SHA-256:**
    - Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. **Target Hash Generation with Nonce:**
    - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
    - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
    - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

Initializing Blockchain

```
import datetime
import hashlib
import json
class Blockchain:
    def __init__(self):
        self.chain = []
        self.create_block(proof = 1, previous_hash = '0')
    def create_block(self, proof, previous_hash):
        block = {'index': len(self.chain) + 1,
                 'timestamp': str(datetime.datetime.now()),
                 'proof': proof,
                 'previous_hash': previous_hash}
        self.chain.append(block)
        return block
    def get_previous_block(self):
        return self.chain[-1]
    def proof_of_work(self, previous_proof):
        new_proof = 1
        check_proof = False
        while check_proof is False:
            hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] == '0000':
                check_proof = True
            else:
                new_proof += 1
        return new_proof
    def hash(self, block):
        encoded_block = json.dumps(block, sort_keys = True).encode()
        return hashlib.sha256(encoded_block).hexdigest()
    def is_chain_valid(self, chain):
        previous_block = chain[0]
        block_index = 1
        while block_index < len(chain):
            block = chain[block_index]
            if block['previous_hash'] != self.hash(previous_block):
                return False
            previous_proof = previous_block['proof']
            proof = block['proof']
            hash_operation = hashlib.sha256(str(proof**2 - previous_proof**2).encode()).hexdigest()
            if hash_operation[:4] != '0000':
                return False
            previous_block = block
```

```
            block_index += 1
        return True
```

Webpage

```
import os
if not os.path.exists('templates'):
    os.makedirs('templates')

index_html_content = """
<!DOCTYPE html>
<html>
<head>
    <title>Blockchain Ledger - Ved & Nikhil</title>
    <style>
        body { font-family: 'Segoe UI', sans-serif; margin: 40px; background: #eceff1; }
        .container { background: white; padding: 30px; border-radius: 12px; box-shadow: 0 4px 20px
rgba(0,0,0,0.1); max-width: 900px; margin: auto; }
        .block { background: #fdfdfd; padding: 15px; margin: 15px 0; border-radius: 8px; border: 1px
solid #dee2e6; border-left: 6px solid #007bff; }
        .btn { padding: 12px 24px; background: #007bff; color: white; text-decoration: none;
border-radius: 6px; font-weight: bold; }
        .hash-text { font-family: monospace; color: #d63384; word-break: break-all; font-size: 0.9em; }
        .meta { color: #6c757d; font-size: 0.9em; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Blockchain Ledger</h1>
        <p><strong>Miners:</strong> Ved & Nikhil</p>
        <p style="color: #28a745;">{{ message if message }}</p>
        <div style="margin-bottom: 20px;">
            <a href="/mine_block" class="btn">Mine New Block</a>
            <a href="/is_valid" class="btn" style="background: #28a745;">Check Validity</a>
        </div>
        {% for block in chain %}
        <div class="block">
            <p><strong>Block #{{ block.index }}</strong> ({{ block.timestamp }})</p>
            <p class="meta">Miner: {{ block.miner }} | Transactor: {{ block.transactor }}</p>
            <p class="meta">Current Hash: <span class="hash-text">{{ block.hash }}</span></p>
            <p class="meta">Prev Hash: <span class="hash-text">{{ block.previous_hash }}</span></p>
        </div>
        {% endfor %}
    </div>
</body>
</html>
"""
with open('templates/index.html', 'w') as f:
    f.write(index_html_content)
```

Flask Application

```
from flask import Flask, render_template, request, redirect, url_for
import threading

app = Flask(__name__)
blockchain = Blockchain()

@app.route('/')
def home():
    msg = request.args.get('message')
    return render_template('index.html', chain=blockchain.chain, length=len(blockchain.chain),
message=msg)

@app.route('/mine_block')
def mine():
    prev = blockchain.get_previous_block()
    proof = blockchain.proof_of_work(prev['proof'])
    block = blockchain.create_block(proof, prev['hash'])
    return redirect(url_for('home', message=f'Block {block["index"]} mined by Ved!'))

@app.route('/is_valid')
def valid():
    is_v = blockchain.is_chain_valid(blockchain.chain)
    return redirect(url_for('home', message='Chain is Valid!' if is_v else 'Chain is Invalid!'))

if not any(t.name == 'FlaskThread' for t in threading.enumerate()):
    threading.Thread(target=lambda: app.run(host='0.0.0.0', port=5000), name='FlaskThread',
daemon=True).start()
    print('Server initialized.')
```


```
from flask import Flask, render_template, request, redirect, url_for
import threading

app = Flask(__name__)
blockchain = Blockchain()

@app.route('/')
def home():
    msg = request.args.get('message')
    return render_template('index.html', chain=blockchain.chain, length=len(blockchain.chain),
message=msg)

@app.route('/mine_block')
def mine():
    prev = blockchain.get_previous_block()
    proof = blockchain.proof_of_work(prev['proof'])
    block = blockchain.create_block(proof, prev['hash'])
```

```
    return redirect(url_for('home', message=f'Block {block["index"]} mined by Ved!'))

@app.route('/is_valid')
def valid():
    is_v = blockchain.is_chain_valid(blockchain.chain)
    return redirect(url_for('home', message='Chain is Valid!' if is_v else 'Chain is Invalid!'))

if not any(t.name == 'FlaskThread' for t in threading.enumerate()):
    threading.Thread(target=lambda: app.run(host='0.0.0.0', port=5000), name='FlaskThread',
daemon=True).start()
    print('Server initialized.')
```